

Spec Driven Development

SDD adaptado a Flutter + Supabase con OpenSpec, integración FADER y referencia para pruebas de y trazabilidad.

GUÍA BASE

**Guía SDD-equipos-
ágiles.pdf**

EXTRAS DEL MÓDULO

**OpenSpec · Flutter ·
Supabase**

Spec-Driven

OpenSpec

Flutter

Supabase

Clean Architecture

Guía completa — 23 Spec Driven Development

1. OpenSpec — Guía práctica

1.1 ¿Qué es OpenSpec?

OpenSpec es un framework de especificación por contexto para agentes de IA que elimina el 80-90% de tokens en conversaciones de larga distancia. Crea especificaciones en lenguaje natural en el directorio `.openspec/` que los agentes leen una sola vez y luego consultan selectivamente.

Por qué SDD no es suficiente solo: SDD define *cómo* trabajar con specs. OpenSpec define *cómo almacenarlas y consultarlas* de forma que los agentes de IA no reinicien el contexto cada vez.

Stickers: openspec.dev · Fission-AI · MIT · 65.7k ★ en GitHub.

1.2 Instalación

```
# npm global (recomendado)
npm install -g @fission-ai/openspec@latest

# Verificar instalación
openspec --version
```

1.3 Comandos principales

Comando	¿Qué hace?	Cuándo usarlo
<code>openspec init</code>	Crea <code>.openspec/</code> + plantilla base	Al inicio del proyecto
<code>openspec propose</code>	Análisis profundo del proyecto, sugiere specs	Proyecto nuevo o importación
<code>openspec exec</code>	Lee specs + ejecuta una tarea (punto de entrada principal)	Diario, antes de cada tarea
<code>openspec status</code>	Muestra specs cargadas, líneas, contexto usado	Diagnosticar contexto inflado
<code>openspec scan</code>	Chequeo de salud del proyecto	Cada 2-3 commits
<code>openspec compress</code>	Opcional: reduce specs para chats con límite	Contextos ajustados
<code>openspec doctor</code>	Check de salud general	Al inicio de sesión
<code>openspec hub</code>	Explorar specs del marketplace	Descubrir nuevas specs
<code>openspec sync</code>	Importar specs desde el hub	Actualizar specs compartidas

1.4 Workflow típico (OpenSpec + SDD)

```
1. openspec init
2. openspec propose           # genera specs candidatas
3. revisar y refinar specs
4. openspec exec " tarea X "  # ejecuta con contexto optimizado
5. openspec status           # verificar uso de contexto
6. openspec scan             # chequeo de integridad
7. repite desde 4
```

1.5 Ejemplo: crear especificación en Flutter

```
# Asumiendo que ya ejecutaste openspec init y propose
openspec exec "Crear el entity User de la feature auth en la capa domain"

# Resultado esperado:
# - Lee domain-spec.md (reglas de la capa domain)
# - Lee architecture-spec.md (clean architecture bindings)
# - Crea lib/features/auth/domain/entities/user.dart
# - Crea test/features/auth/domain/entities/user_test.dart
```

1.6 Slash commands para agentes de IA

OpenSpec expone 10 slash commands para usar en chats con agentes:

Comando	Función
<code>/openspec-list</code>	Lista specs cargadas y contexto usado
<code>/openspec-info</code>	Detalle de una spec específica
<code>/openspec-on</code>	Activa una spec para la sesión
<code>/openspec-off</code>	Desactiva una spec
<code>/openspec-status</code>	Resumen de estado
<code>/openspec-suggest</code>	Sugiere specs relevantes para una tarea
<code>/openspec-validate</code>	Valida que el código cumple las specs
<code>/openspec-audit</code>	Audita specs contra el código actual
<code>/openspec-upgrade</code>	Mejora specs existentes
<code>/openspec-compress</code>	Reduce tamaño de specs

1.7 OpenSpec vs Spec Kit vs Kiro

Aspecto	OpenSpec	Spec Kit	Kiro (AWS)
Enfoque	Contexto optimizado para agentes	Especificaciones para IA (general)	Specs + agente autónomo
Archivo de config	<code>.openspec/</code>	<code>.spec/</code>	<code>.kiro/</code>
Ejecución	<code>openspec exec</code>	Manual / integrado	En el IDE
Marketplace	Sí (Hub)	No	No
Gratuito	Sí (MIT)	Sí	Sí tier inicial

2. SDD en Flutter — Adaptación completa

2.1 ¿Qué es SDD?

Spec Driven Development (SDD) es una metodología donde **las especificaciones se escriben primero** y el código se genera para cumplirlas. El objetivo: (*rigidez a cambio*) se detecta en specs antes de que se convierta en bugs. La guía completa está en `Guia-SDD-equipos-agiles.pdf`. Este capítulo adapta los conceptos a Flutter + Supabase + Clean Architecture.

2.2 SDD en el contexto de Clean Architecture

Cada capa de Clean Architecture tiene un tipo de spec diferente:

Capa	Tipo de spec	Ejemplo
Domain	Spec de entidad	"El User tiene id UUID, email, displayName; no es nullable"
Domain	Spec de use case	"Login recibe email+password, retorna User o AuthException"
Domain	Spec de repositorio (interfaz)	"LoginRepository.login() es el único punto de acceso a auth"
Infrastructure	Spec de datasource	"SupabaseDatasource usa client.auth.signInWithPassword()"
Infrastructure	Spec de repository impl	"RepositoryImpl delega a Datasource y mapea excepciones"
Presentation	Spec de cubit	"AuthCubiti tiene estados: initial, loading, authenticated, error"
UI	Spec de página	"LoginEmailPage usa listener_builder, muestra loading en Submit"

2.3 Reglas de los boundaries (Capítulo 6 — Guía SDD)

Los boundaries definen qué puede y qué no puede hacer cada capa:

Boundary	Regla en Flutter+Supabase
Domain → Infrastructure	Domain define interfaces; Infrastructure las implementa. Nunca al revés.
Domain → Data	Domain no importa paquetes de Supabase; solo define contratos.
Infrastructure → Supabase	Solo datasources tocan Supabase client. Repositories solo coordinan.
Presentation → Domain	Cubits usan UseCases. Nunca usan Datasources directamente.
UI → Presentation	Páginas usan BlocBuilder/BlocListener. Nunca lógica de negocio.

2.4 Especificaciones EARS para Flutter

EARS (Easy Approach to Requirements Syntax) adapta SDD a requisitos verificables:

Patrón EARS	Ejemplo Flutter
WHILE [condición] the system shall [comportamiento]	WHILE el usuario no está autenticado, the system shall redirigir a /login
IF [trigger] the system shall [comportamiento]	IF el email es inválido, the system shall mostrar error en el campo
WHERE [componente] the system shall [comportamiento]	WHERE AuthCubiti, the system shall emitir AuthError en excepción
WHEN [evento] the system shall [comportamiento]	WHEN se presiona Submit, the system shall llamar loginUseCase
GIVEN [contexto] WHEN [acción] the system shall [resultado]	GIVEN usuario registrado, WHEN presiona "Olvidé contraseña", the system shall enviar email de reset

2.5 Plantilla de spec para Flutter+Supabase

```
# FEATURE: auth
## Componente: UserEntity

### Tipo
Especificación de entidad (Domain layer)

### Requisitos
- WHERE UserEntity, the system shall tener campos: id (UUID), email (String, unique), displayName (String), createdAt (DateTime)
- WHERE UserEntity, the system shall NO tener campos nullable excepto displayName
- WHERE UserEntity, the system shall implement fromJson/toJson con serialización correcta

### Límites
- UserEntity NO debe importar nada de Supabase ni de Firebase
- UserEntity NO debe tener lógica de negocio (solo datos)
- UserEntity debe ser immutable (usar freezed si aplica)

### Validación
- Test: UserEntity.fromJson({'id': '...'}) retorna instancia válida
- Test: UserEntity.fromJson({'email': null'}) lanza error de validación
- Test: user == user2 (misma data) retorna true
```

2.6 Mapeo Postgres → Dart (Supabase)

Para cada tabla de Supabase, genera la spec de entity automáticamente:

```
# Tabla Supabase: users
# Columnas: id (uuid, PK), email (text, unique), display_name (text), created_at (timestampz)

## Spec resultante:
- Entity: User con id (String), email (String), displayName (String), createdAt (DateTime)
- Model: UserModel.fromJson() mapea snake_case → camelCase
- Datasource: SupabaseClient.from('users').select().eq('id', userId)
```

3. Integración SDD + FADER (Módulo 02)

3.1 Comparación SDD vs FADER

Aspecto	SDD (Guía SDD)	FADER (Módulo 02)
Enfoque	Specs primero, código después	Alcance → Diseño → Specs → Build
Phases	4: Diseño, Desarrollo, Validación, Mantenimiento	6: Alcance, FADER, Contratos, Flujo, Backend, Criterios
Specifies	Especificaciones en lenguaje natural + EARS	FADER document + diagramas + templates
Agente IA	OpenSpec (lectura optimizada de contexto)	Agentes de scaffold (clean-arch-feature)
Output esperado	Specs aprobadas + código que las cumple	Feature scaffold completa + wiring

3.2 Flujo integrado (SDD + FADER + Flutter)

Paso	FADER	SDD/OpenSpec	Output
1	Alcance	<code>openspec propose</code>	Alcance + specs candidatas
2	FADER	Refinar specs con OpenSpec	FADER document + specs aprobadas

Paso	FADER	SDD/OpenSpec	Output
3	Contratos	Especs EARS por componente	Interfaces + entities con requisitos
4	Flujo	Spec de UI + spec de Cubit	Diagrama de estados + wiring
5	Backend	Spec de datasource + SQL	SQL migration + Supabase client config
6	Criterios	<code>openspec validate</code>	Tests + validación de specs
7	Scaffold	<code>openspec exec " build "</code>	Feature completa generada

3.3 Ejemplo integrado: Login con Supabase Auth

Fase 1 — Alcance (FADER)

```
# Alcance Login con Supabase
- Feature: auth/login
- Componentes: LoginPage, AuthCubit, LoginUseCase, AuthRepository, SupabaseDatasource
- Supabase: tabla profiles (id FK auth.users, display_name)
- Screenshots: /assets/login-wireframe.png
```

Fase 2 — Specs EARS (SDD)

```
## AuthCubiti
- WHEN usuario presiona Submit Y email es válido, the system shall emitir AuthLoading
- WHEN login retorna User, the system shall emitir AuthAuthenticated(user)
- WHEN login lanza AuthException, the system shall emitir AuthError(message)
- WHEN login lanza excepción genérica, the system shall emitir AuthError('Error inesperado')

## LoginEmailPage
- WHERE LoginEmailPage, the system shall usar listener_builder para AuthCubit
- WHILE AuthLoading, the system shall mostrar CircularProgressIndicator
- WHEN AuthError, the system shall mostrar SnackBar con el mensaje

## SupabaseDatasource
- WHERE SupabaseDatasource, the system shall usar client.auth.signInWithPassword()
- WHEN signInWithPassword lanza AuthException, the system shall relanzar AuthException
```

Fase 3 — Build (Scaffold)

```
# Ejecutar scaffold (delegado a clean-arch-feature)
# Genera:
# - auth/domain/entities/user.dart
# - auth/data/models/user_model.dart
# - auth/data/datasources/supabase_auth_datasource.dart
# - auth/data/repositories/auth_repository_impl.dart
# - auth/domain/repositories/auth_repository.dart
# - auth/domain/usecases/login_usecase.dart
# - auth/presentation/cubit/auth_cubit.dart
# - auth/presentation/pages/login_email_page.dart
```

4. Referencia rápida — Cheat sheet

4.1 Las 4 fases de SDD

Fase	Objetivo	Output esperado	Tool principal
Diseño	Definir qué construir	Specs aprobadas	FADER + OpenSpec
Desarrollo	Generar código que cumpla specs	Feature completa	Scaffold + Agentes

Fase	Objetivo	Output esperado	Tool principal
Validación	Verificar que specs se cumplen	Tests + Validación	openspec validate
Mantenimiento	Actualizar specs al cambiar	Specs actualizadas	openspec scan

4.2 Los 3 Gates de aprobación

Gate	Cuándo	Criterio
Gate 1: Diseño	Antes de escribir código	Specs aprobadas por stakeholder
Gate 2: Implementación	Antes de merge a main	Código cumple specs + tests pasan
Gate 3: Validación	Antes de release	<code>openspec validate</code> pasa limpio

4.3 OpenSpec — Comandos esenciales

```
openspec init      # Crear .openspec/
openspec propose   # Generar specs candidatas
openspec exec "task" # Ejecutar tarea con contexto
openspec status    # Ver uso de contexto
openspec scan      # Chequeo de salud
openspec validate  # Validar código contra specs
```

4.4 Anti-patrones en SDD

Anti-patrón	Consecuencia	Solución
Escribir specs vagas	Código ambiguo, bugs silenciosos	Usar EARS: patrón específico
Saltar Gate 1	Features que no matchean necesidades	Siempre aprobar specs antes de código
Specs monolíticas	Difícil de mantener, hard de testear	1 spec por componente + boundaries
No actualizar specs al cambiar	Specs desactualizadas, código roto	Actualizar specs en cada PR relevante
Usar OpenSpec sin SDD	Contexto optimizado pero sin methodology	OpenSpec es tool, SDD es methodology

4.5 Proporcionalidad de specs

Tipo de feature	Profundidad de spec	Ejemplo
CRUD simple	Básica (1 spec por componente)	Formulario de contacto
Feature core	Completa (EARS + boundaries + tests)	Login con Supabase Auth
Integración externa	Alta (mocks + contratos + validación)	Pago con Stripe
Fix de bug	Mínima (solo spec del componente)	Corregir cálculo de IVA